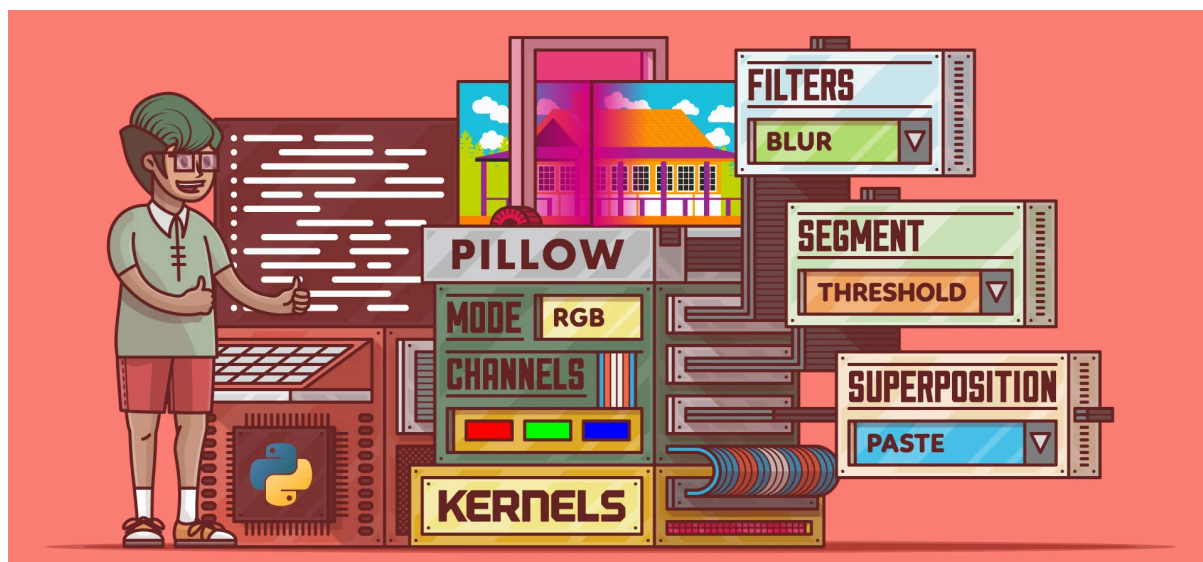


# Project Hands V3



## Indice

|                                                   |          |
|---------------------------------------------------|----------|
| <b>PROGETTO VISIONE ARTIFICIALE.....</b>          | <b>1</b> |
| Introduzione.....                                 | 3        |
| Possibili miglioramenti.....                      | 3        |
| Problemi attuali.....                             | 3        |
| Applicazioni possibili.....                       | 4        |
| Librerie utilizzate.....                          | 4        |
| Spiegazione del codice.....                       | 5        |
| Configurazione iniziale.....                      | 5        |
| Configurazione della mano.....                    | 5        |
| Landmarks sulla mano.....                         | 6        |
| Loop principale.....                              | 6        |
| Pre-processing.....                               | 6        |
| Rilevazione mano ed estrapolazione landmarks..... | 7        |
| Coordinate su schermo.....                        | 8        |
| Visualizzazione e uscita.....                     | 8        |
| Benchmark e tempi di risposta.....                | 9        |
| Diagramma degli eventi.....                       | 9        |



# ***Introduzione***

In questo progetto ho sviluppato un sistema di controllo del cursore del mouse tramite il riconoscimento dei gesti della mano, utilizzando la libreria MediaPipe per il tracciamento e PyAutoGUI per l'interazione con il sistema operativo. L'obiettivo principale è stato quello di esplorare le potenzialità della computer vision, realizzando un'interfaccia naturale e senza contatto che consenta il controllo fisico del computer ed attraverso semplici movimenti della mano è possibile controllare quest'ultimo. Il sistema è stato progettato per essere utilizzato in tempo reale e senza hardware aggiuntivo, sfruttando semplicemente una webcam standard.

Il funzionamento è il seguente:

Con il polso chiuso si muove il cursore, solo l'indice alzato corrisponde al click sinistro, solo il dito medio alzato corrisponde a click destro. Per trascinare basta muovere il cursore su quello che si vuole selezionare e tenere alzato il dito indice e muovere il cursore per trascinare, per rilasciare basta abbassare l'indice.

## ***Possibili miglioramenti***

Attualmente, il sistema funziona in modo discreto in condizioni di luce ottimali e con sfondi semplici, ma presenta ancora margini di miglioramento. Una delle direzioni principali su cui lavorare riguarda l'integrazione di filtri per la stabilizzazione dei movimenti, al fine di rendere il controllo del cursore più fluido. Un'altra possibile evoluzione del progetto potrebbe essere l'aggiunta del supporto per più gesti complessi, come il pinch-to-zoom o lo swipe (magari introducendo l'uso della mano sinistra, anche se potrebbe causare un uso più smisurato della complessità di calcolo), oltre alla possibilità di configurare gesture personalizzate. Inoltre, si potrebbe implementare una calibrazione automatica che adatti il comportamento del sistema in base all'ambiente e alla posizione dell'utente.

## ***Problemi attuali***

Uno dei limiti principali riguarda le prestazioni del sistema in ambienti poco illuminati o con sfondi con molti dettagli. In queste condizioni, l'algoritmo di tracciamento risulta meno preciso e può causare comportamenti instabili o movimenti scattosi del cursore. Questo è dovuto alla difficoltà, per il modello, di distinguere correttamente la mano dal contesto. Inoltre, anche piccoli ritardi nella rilevazione dei gesti possono compromettere l'esperienza d'uso, rendendo necessario un miglioramento nella gestione dei frame e dell'ottimizzazione del flusso video.

## ***Applicazioni possibili***

- La possibilità di controllare il computer senza mouse fisico, il che risulta particolarmente utile in ambienti in cui l'igiene è fondamentale, come strutture sanitarie o laboratori.
- Accessibilità per persone con difficoltà motorie.
- Integrazione con videogiochi offrendo un'esperienza utente più immersiva(e inoltre è un sistema molto familiare ai VR in giro, chiaramente molto meno preciso e in 2D invece che 3D).
- Potrebbe essere impiegato in ambito educativo nella Pubblica Amministrazione(uffici, scuole, ecc) per presentazioni o contesti educativi per navigare tra slide o contenuti in modo dinamico e senza contatto.

## ***Librerie utilizzate***

**cv2** OpenCV serve a gestire la webcam e l'elaborazione delle immagini.

**mediapipe** è una libreria di Google per il tracciamento del corpo, viso, mani, ecc.

**pyautogui** serve a simulare l'input del mouse/tastiera.

**numpy** lo utilizzo per interpolare coordinate (mappatura tra video e schermo).

**time** lo utilizzo per gestire il cooldown tra due click.

# ***Spiegazione del codice***

## ***Configurazione iniziale***

```
pyautogui.FAILSAFE = False  
cap = cv2.VideoCapture(1)  
cap.set(3,640)  
cap.set(4,480)
```

*Disattiva il failsafe di PyAutoGUI (che altrimenti blocca tutto se il mouse va nell'angolo in alto a sinistra). Poi apre la webcam (con indice 1) e imposta la grandezza della finestra della webcam.*

## ***Configurazione della mano***

```
mp_hands = mp.solutions.hands  
hands = mp_hands.Hands(  
    max_num_hands=1,  
    min_detection_confidence=0.8,  
    min_tracking_confidence=0.8  
)  
mp_draw = mp.solutions.drawing_utils
```

Prima carica il modulo per il tracciamento delle mani, il valore `max_num_hands=1` considera una sola mano per semplicità, mentre `min_detection_confidence` e `min_tracking_confidence` sono alti apposta per avere maggiore precisione. Infine `mp_draw` serve per disegnare la mano sul frame.

### ***Landmarks sulla mano***

```
def fingers_up(lm_list):
    tips = [8, 12, 16, 20] # Indice, Medio, Anulare, Mignolo
    up = []
    for tip in tips:
        up.append(lm_list[tip][1] < lm_list[tip - 2][1])
    return up
```

Prende la lista dei landmark (ovvero delle coordinate) delle dita e confronta la y del tip con quella della base per ogni dito per capire se è alzato.

### ***Loop principale***

```
while True:
    success, frame = cap.read()
    if not success:
        continue
```

Cattura un frame dalla webcam, se non riesce rientra nel ciclo.

### ***Pre-processing***

```
frame = cv2.flip(frame, 1)
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = hands.process(frame_rgb)
```

Ribalta l'immagine orizzontalmente per avere un effetto specchio, converte da BGR a RGB (perchè MediaPipe usa RGB) ed infine elabora il frame con MediaPipe per trovare la mano.

## *Rilevazione mano ed estrapolazione landmarks*

`if results.multi_hand_landmarks:`

`for hand_landmarks, handedness in zip(results.multi_hand_landmarks, results.multi_handedness):`

`hand_label = handedness.classification[0].label`

`if results.multi_hand_landmarks` controlla se MediaPipe ha trovato almeno una mano nel frame.

`results` è l'oggetto restituito da `hands.process(frame_rgb)`.

`multi_hand_landmarks` è una lista di mani rilevate. Se è `None`, significa che nessuna mano è stata trovata, quindi non entra nel blocco.

`for hand_landmarks, handedness in zip(results.multi_hand_landmarks, results.multi_handedness)` itera sulle mani trovate e sulle loro informazioni (posizione + destra/sinistra). `multi_hand_landmarks` contiene i landmarks (cioè i punti della mano come pollice, indice, ecc.).

`multi_handedness` contiene la classificazione della mano (se è Right o Left).

`zip()` serve per prendere in parallelo i landmark e la classificazione di ciascuna mano.

`hand_label = handedness.classification[0].label` estrae l'etichetta della mano, cioè se è la Right o Left. `handedness` è un oggetto con una lista chiamata `classification`, `classification[0].label` contiene la stringa Right o Left.

Questo mi permette di fare azioni diverse solo se è la mano destra, ad esempio per non confondere le mani se ce ne sono due nel frame.

Quindi, riassumendo, questa parte controlla se c'è almeno una mano nel video, e poi per ciascuna mano estrapola i suoi landmarks e capisce se è la destra o la sinistra, salvandolo nella variabile `hand_label`.

## ***Coordinate su schermo***

```
lm_list = [(int(lm.x * frame.shape[1]), int(lm.y * frame.shape[0])) for lm in
hand_landmarks.landmark]
mp_draw.draw_landmarks(frame, hand_landmarks,
mp_hands.HAND_CONNECTIONS)

up = fingers_up(lm_list)
wrist = lm_list[0]
x_screen = np.interp(wrist[0], (0, frame.shape[1]), (0, screen_width))
y_screen = np.interp(wrist[1], (0, frame.shape[0]), (0, screen_height))

dx = x_screen - prev_x
dy = y_screen - prev_y
```

Calcola la posizione di ogni landmarks, poi disegna la mano sullo schermo ed usa il polso (landmark 0) per controllare il mouse, successivamente mappa le coordinate del polso dal frame al desktop.

## ***Visualizzazione e uscita***

```
cv2.imshow("Hand Controller", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Mostra la finestra con il video e la mano tracciata. Chiude tutto se premi q.



## ***Benchmark e tempi di risposta***

Per una webcam di scarsa qualità con massimo 25 FPS variano tra i 6 e i 7 in fase di applicazione, il tempo per ogni frame è di circa 0.1524 secondi.

## ***Diagramma degli eventi***

